

Edge AI Earbud Supervisor — Phase-Wise Execution Plan

Build spec / implementation roadmap — companion to the project blueprint document. BTech AIML Final Year Project.

How to use this document

The blueprint document covers the *why* and *what* — architecture, motivation, viva defense. This document covers the *how* — concrete phases with tasks, technical parameters, and time estimates, so the project can actually be built and tracked week to week.

Total estimated effort: ~90-100 hours across ~3 months, run alongside DSA prep (not instead of it).

Phase 1 — Data Sourcing & Class Mapping

Estimated time: 10-15 hrs

Goal: build a clean, labeled 3-class audio dataset from existing sources — no manual field recording required for training data.

Source datasets:

- **UrbanSound8K** — car horns, sirens, engine idling, street/drilling noise. Primary source.
- **ESC-50** — broader ambient sounds: rain, wind, silence-adjacent quiet room sounds, hums.
- **Google AudioSet** (optional, larger scope) — YouTube-sourced clips tagged 'siren', 'alarm', 'traffic noise'.

Class mapping (fills the gap the blueprint left open):

Target Class	Source Labels to Pull	Approx. Samples Ne
Class A — Continuous Low-Freq Hum (→ Max ANC Mode)	engine_idling, jackhammer, drilling, air_conditioner, street_music (low-freq subset)	300-500 clips
Class B — Transient Danger Profile (→ Safety Transparency Mode)	siren, car_horn, gun_shot (ESC-50), scream/shout categories if available	300-500 clips
Class C — Low-Amplitude Quiet (→ Low-Power Standby Mode)	silence/near-silence clips, ESC-50 'quiet room' equivalents, low-RMS filtered background clips	300-500 clips

Tasks:

- Download UrbanSound8K + ESC-50, inspect label taxonomy against the 3-class map above.
- Write a filtering script to bucket source labels into Class A / B / C, discard irrelevant classes.
- Balance classes (roughly equal sample counts) to avoid biased training.
- Hold out ~15-20 real-mic recordings per class (phone-emitted, laptop-mic captured) — needed later for live demo validation, since the entire pitch is 'raw PCM captures what stock filtering misses'.
- Split: 70% train / 15% validation / 15% test, stratified by class.

Phase 2 — Signal Processing Pipeline

Estimated time: 10-12 hrs

Goal: convert raw audio clips into the 2D spectrogram inputs the CNN will consume.

Fixed parameters (fills the buffer/windowing gap):

Parameter	Value	Notes
Sample rate	16,000 Hz	Standard for speech/environmental audio, keeps compute light
Window size	200 ms (analysis block)	Matches blueprint's stated inference cadence
Frame overlap / hop length	50% overlap, hop = 512 samples	Smooths classification, avoids boundary artifacts
Feature type	Mel-spectrogram, 64 mel bands	Slightly better than raw MFCC for CNNs
FFT window (n_fft)	1024	librosa.stft default-adjacent, fine resolution
Normalization	Per-clip min-max or log-scale (dB)	librosa.power_to_db, then scale to [0,1]

Tasks:

- Write librosa-based conversion function: raw wav → mel-spectrogram → fixed-size 2D array (pad/truncate to constant shape, e.g. 64 x 63).
- Batch-convert entire dataset, cache as .npy arrays to avoid recomputing on every training run.
- Visualize a few spectrograms per class to sanity-check separability before training (siren should look visually distinct from hum).

Phase 3 — Model Design, Training & Evaluation

Estimated time: 20-25 hrs

Goal: train a lightweight CNN classifier and evaluate it against the metric the blueprint specifies — minimizing false negatives on the danger class.

Model architecture (fills the 'lightweight CNN' gap with real numbers):

- Input: 64 x 63 x 1 mel-spectrogram array.
- Conv2D (16 filters, 3x3, ReLU) → MaxPool(2x2)
- Conv2D (32 filters, 3x3, ReLU) → MaxPool(2x2)
- Conv2D (64 filters, 3x3, ReLU) → GlobalAveragePooling2D (keeps param count low, avoids huge Dense layer)
- Dense (32, ReLU) → Dropout(0.3) → Dense(3, Softmax)
- Target: under 100K parameters total — keeps it genuinely 'edge-deployable', not just called that.

Training config:

Hyperparameter	Value
Optimizer	Adam, lr = 1e-3 (with ReduceLROnPlateau)

Loss	Categorical cross-entropy
Batch size	32
Epochs	30-50 (early stopping on val loss, patience=5)
Class weighting	Weighted loss favoring Class B (danger) to bias against false negatives

Evaluation plan:

- Report accuracy, but treat it as secondary — headline metrics are per-class Precision, Recall, F1.
- Confusion matrix — explicitly check Class B (danger) false-negative rate, this is the core safety claim of the whole project.
- Target: Recall on Class B $\geq 90\%$ even if it costs some precision (a false alarm is acceptable; missing a real siren is not).
- Latency benchmark (fills the blueprint's missing number): measure inference time per 200ms window on the target laptop CPU — target under 50ms per inference, well inside the 200-300ms budget the blueprint claims.

Phase 4 — Live Capture & Raw PCM Pipeline

Estimated time: 12-15 hrs

Goal: implement the raw-PCM microphone capture described in the blueprint (Section 4) and connect it to the trained model for real-time inference.

Tasks:

- Set up PyAudio or sounddevice stream with raw input flag, bypassing OS-level ENC/AGC where the driver allows it.
- Implement a rolling 200ms buffer with the same hop/overlap settings as training preprocessing — mismatch here is the most common real-world bug.
- Run the held-out real-mic recordings from Phase 1 through the live pipeline first, before testing with live sounds — isolates model bugs from pipeline bugs.
- Validate against the blueprint's core physics claim: confirm a distant (10-15m) phone-emitted horn/siren is picked up and classified correctly; document actual vs claimed detection range honestly for the report.

Phase 5 — Mode Routing Logic

Estimated time: 8-10 hrs

Goal: implement the 3-mode switching script (blueprint Section 3) with real threshold/debounce logic — not just a bare argmax on classifier output.

Tasks:

- Argmax on softmax output selects the candidate mode per 200ms window.
- Add a debounce/smoothing rule (e.g. require the same class for 2-3 consecutive windows before switching) to avoid audio mode flickering on noisy boundary frames.

- Exception: Safety Transparency Mode should NOT be debounced — trigger instantly on a single high-confidence danger detection, since latency here is a safety issue.
- Implement the 3 output behaviors per blueprint Section 3 table: brown/pink noise playback (ANC), direct mic passthrough (Safety), reduced sampling rate loop (Standby).

Important clarification — signal vs. analogy:

The masking noise played in Max ANC Mode is **not** performing real noise cancellation — it is an audible stand-in so a human watching the demo can perceive that a mode-switch decision fired. The actual output of the system at this point is a discrete decision signal (e.g. 'Class A detected → trigger Max ANC state'). On real ANC hardware (earbuds/phones with existing ANC controllers), this decision would be sent as a command/API call to that hardware's own ANC system, not as a played-back sound. Do not present the masking noise as the project's noise-cancellation mechanism in the report or viva — the project's contribution is the classification-and-decision layer, not audio cancellation itself.

Path to production (practicality answer for viva):

This system is designed as a decision layer that would sit on top of existing ANC hardware, not replace or reimplement it. On a real device (earbuds/phones already equipped with ANC and transparency-mode hardware, e.g. AirPods, Sony, Bose), the classifier's output would call that device's existing ANC/transparency control API instead of running a masking-noise or mic-passthrough script. The pipeline is: **classify** → **decide** → **command existing hardware** — directly portable to production without needing to build any ANC hardware.

Phase 6 — Demo, Report & Deployment Polish

Estimated time: 15-20 hrs

Goal: package the project so it's demoable live and reads well on a resume/GitHub — this is the differentiator vs a notebook-only submission.

Tasks:

- Live demo script: phone plays test sounds at set distances, laptop mic + pipeline classifies and switches modes visibly (console/UI indicator).
- GitHub repo: clean README with architecture diagram, metrics table (precision/recall/F1 per class), sample spectrograms, short demo GIF/video.
- Final report: merge blueprint's conceptual sections with this phase plan's actual results (real numbers, not projected ones) and the viva defense answers.
- Do NOT attempt the embedded/ARM Cortex-M deployment this semester — mention only as future work, per blueprint Section 5, Q2 answer.

Scope Guardrails

Explicit cuts to keep this finishable alongside DSA prep:

- No physical hardware/PCB build — laptop simulation only, as blueprint specifies.
- No embedded/microcontroller deployment — mention as future work only.
- No custom audio dataset recording as the primary data source — Kaggle/UrbanSound8K/ESC-50 first, real-mic recording limited to demo validation samples only.

- If timeline slips, cut Phase 6 polish depth before cutting Phase 3 evaluation rigor — a working, honestly-evaluated model matters more than a polished GitHub page.

Companion document to: Edge AI Earbud Project Blueprint (Smart Context-Aware Acoustic Supervisor Framework)